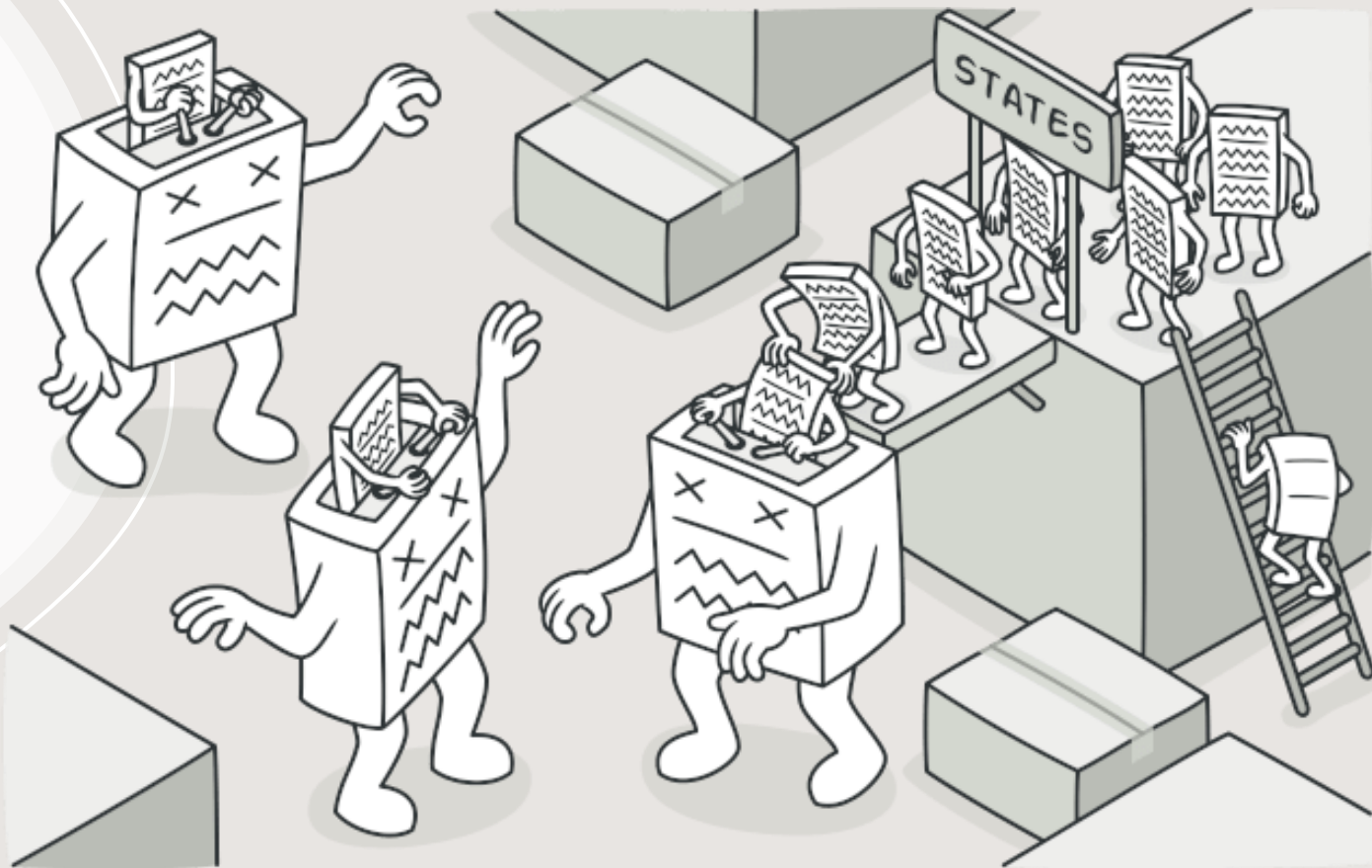


State

Stav



Hlavní idea

State (Stav) je behaviorální návrhový vzor (vzor související s chováním), který umožňuje objektu změnit své chování, když se změní jeho vnitřní stav. Vypadá to, jako by objekt změnil svou třídu.

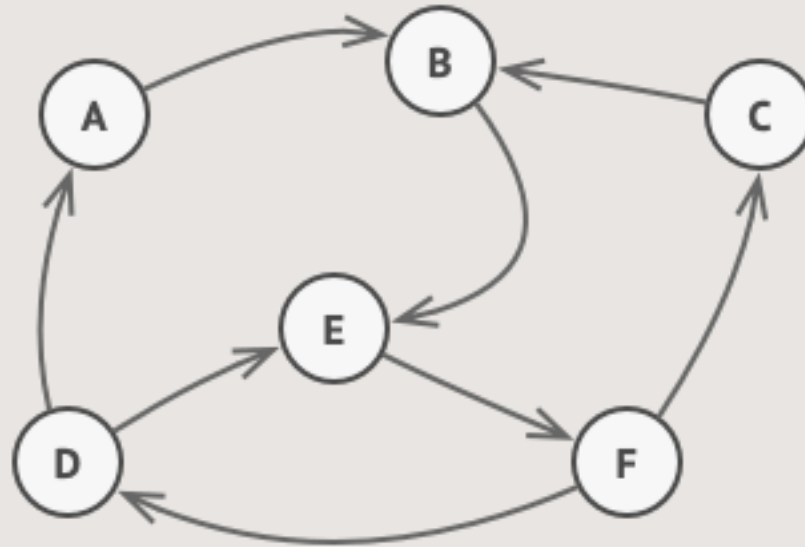


Návrhový vzor State umožňuje objektu změnit své chování, které je závislé na jeho stavu. Nahrazuje složité větvení uvnitř objektu a chování pro konkrétní stavy odděluje do samostatných tříd.



Problém

Vzor stavu úzce souvisí s konceptem konečného automatu, tzv.
finite state machine





Problém

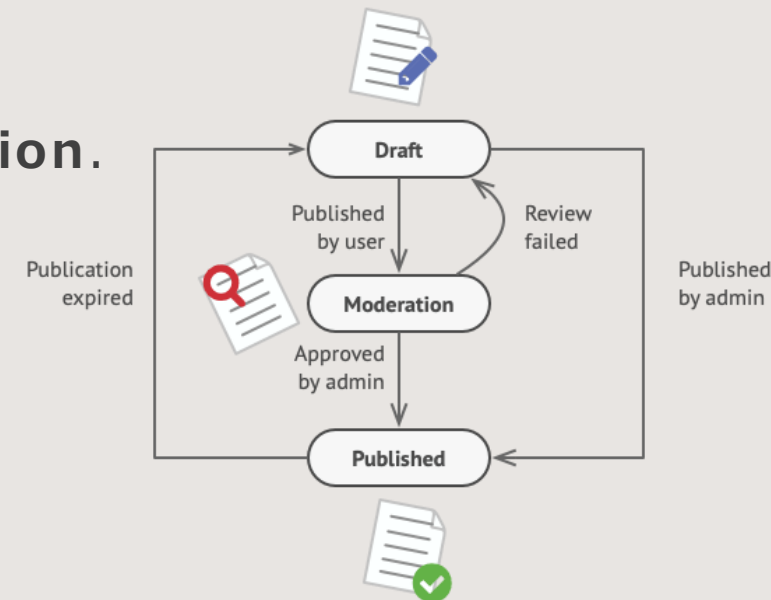
Hlavní myšlenkou je, že v každém daném okamžiku existuje konečný počet stavů, ve kterých se program může nacházet. V rámci jakéhokoli jedinečného stavu se program chová odlišně a v programu lze okamžitě přejít z jednoho stavu do druhého. V závislosti na aktuálním stavu se však program může nebo nemusí přepnout do určitých jiných stavů. Tato pravidla přepínání, nazývané **přechody** (*transitions*), jsou také konečné a předem určené.



Problém

Tento přístup můžete použít i na objekty. Představte si, že máme třídu **Document**. Dokument může být v jednom ze tří stavů: **Draft** (Koncept), **Moderation** (Moderování) a **Published** (Publikováno). Metoda publikování dokumentu funguje v každém stavu trochu jinak:

- Ve stavu **Draft** přesune dokument do stavu **Moderation**.
- V režimu **Moderation** dokument zveřejní (do stavu **Published**), ale pouze pokud je aktuální uživatel správcem.
- Ve stavu **Published** se nestane nic





Problém

Stavové automaty jsou obvykle implementovány se spoustou podmíněných operátorů (if nebo switch), které vybírají vhodné chování v závislosti na aktuálním stavu objektu. Obvykle je tento „stav“ pouze souborem hodnot polí objektu. I když jste o konečných strojích nikdy předtím neslyšeli, pravděpodobně jste stav alespoň jednou implementovali.

```
class Document is
    field state: string
    // ...
    method publish() is
        switch (state)
            "draft":
                state = "moderation"
                break
            "moderation":
                if (currentUser.role == 'admin')
                    state = "published"
                break
            "published":
                // Do nothing.
                break
    // ...
```



Problém

Největší slabina stavového automatu založeného na podmíněnosti se ukáže, jakmile do třídy **Document** začneme přidávat další a další stavy a stavově závislé chování. Většina metod bude obsahovat komplexní podmínky, které vyberou správné chování metody podle aktuálního stavu. Kód je však velmi obtížné udržovat, protože jakákoli změna logiky přechodu může vyžadovat změnu podmínek stavu v každé metodě.

Problém má tendenci se zvětšovat, jak se projekt vyvíjí. Je docela obtížné předvídat všechny možné stavy a přechody ve fázi návrhu. Původní subtilní automat vytvořený s omezenou sadou podmínek se tak může časem rozrůst v robustní.



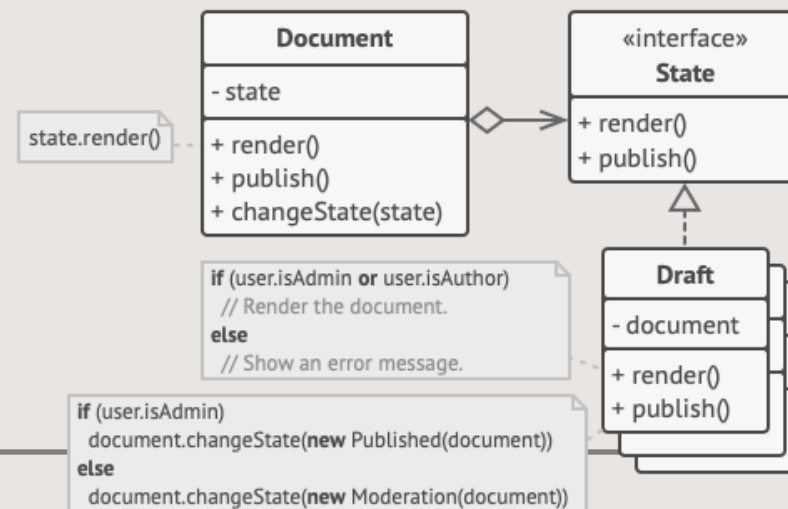
Řešení

Návrhový vzor **Stav** doporučuje vytvoření nové třídy pro všechny možné stavy objektu a do těchto tříd implementovali všechny stavově specifické chování.

Namísto samostatné implementace všech chování v původním objektu, nazývaný kontext, uchovává odkaz na jeden z objektů stavu, který představuje jeho aktuální stav, a deleguje veškerou práci související se stavem na tento objekt.



Řešení



Chcete-li převést **kontext** do jiného stavu, nahraďte objekt aktivního stavu jiným objektem, který tento nový stav představuje. To je možné pouze v případě, že všechny stavové třídy sledují stejné rozhraní a samotný kontext pracuje s těmito objekty prostřednictvím tohoto **rozhraní**.

Tato struktura může vypadat podobně jako vzor **Strategy**, ale je tu jeden klíčový rozdíl. Ve vzoru **State** si jednotlivé stavy mohou být navzájem vědomy a iniciovat přechody z jednoho stavu do druhého, zatímco vzor **Strategy** o sobě téměř nikdy nevědí.



Analogie v reálném světě

Tlačítka a přepínače ve vašem smartphonu se chovají odlišně v závislosti na aktuálním stavu zařízení:

- Když je telefon odemčený, stisknutí tlačítek vede k provádění různých funkcí.
- Když je telefon uzamčen, stisknutím libovolného tlačítka se dostanete na obrazovku odemknutí.
- Když je nabití telefonu nízké, stisknutím libovolného tlačítka zobrazíte obrazovku nabíjení.



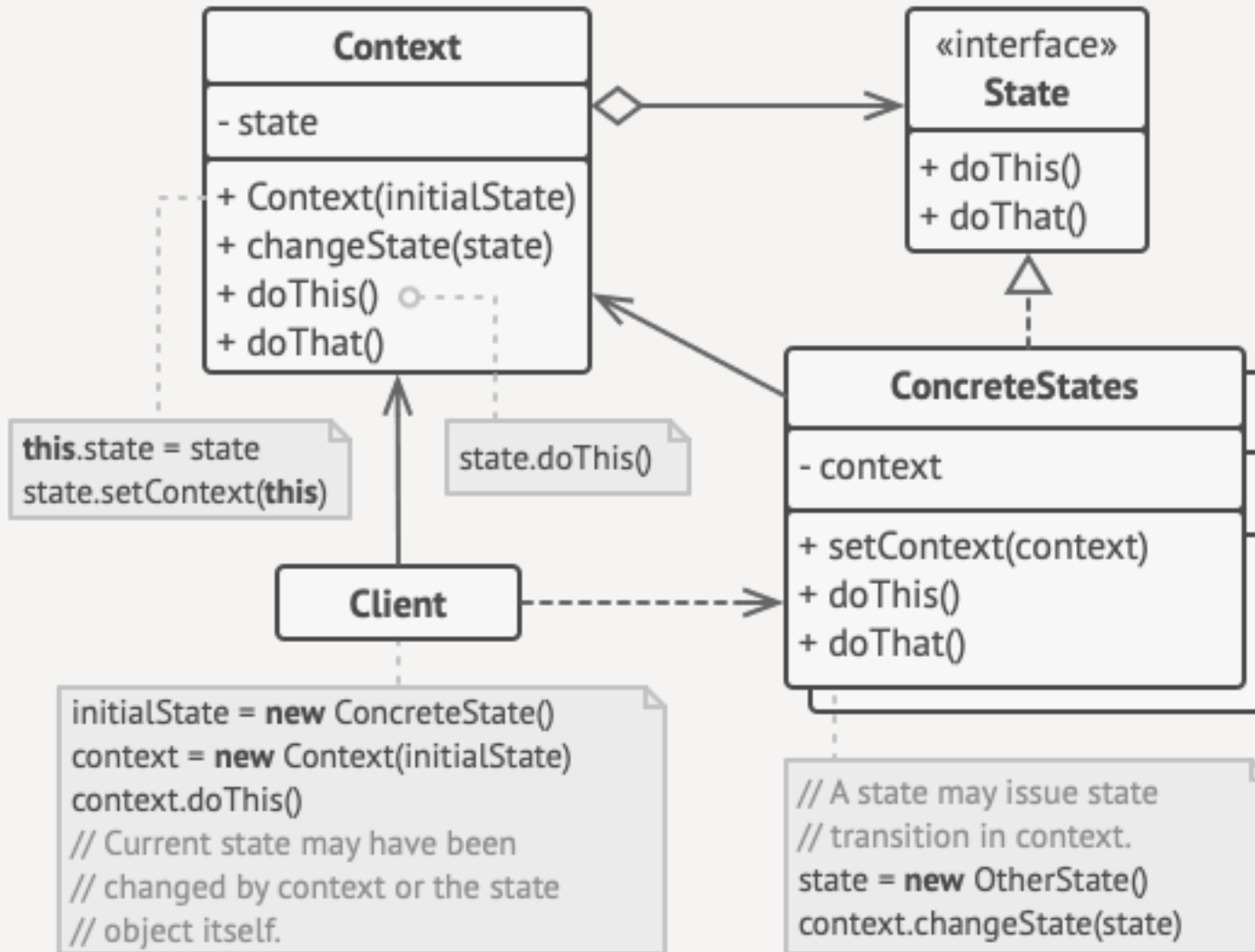
Výhody a nevýhody



- Princip jednotné odpovědnosti. Uspořádejte kód související s konkrétními stavy do samostatných tříd.
- Princip otevřený/uzavřený. Zaveďte nové stavy, aniž byste změnili existující třídy stavů nebo kontext.
- Zjednodušte kód kontextu odstraněním objemných podmíněných stavových automatů.

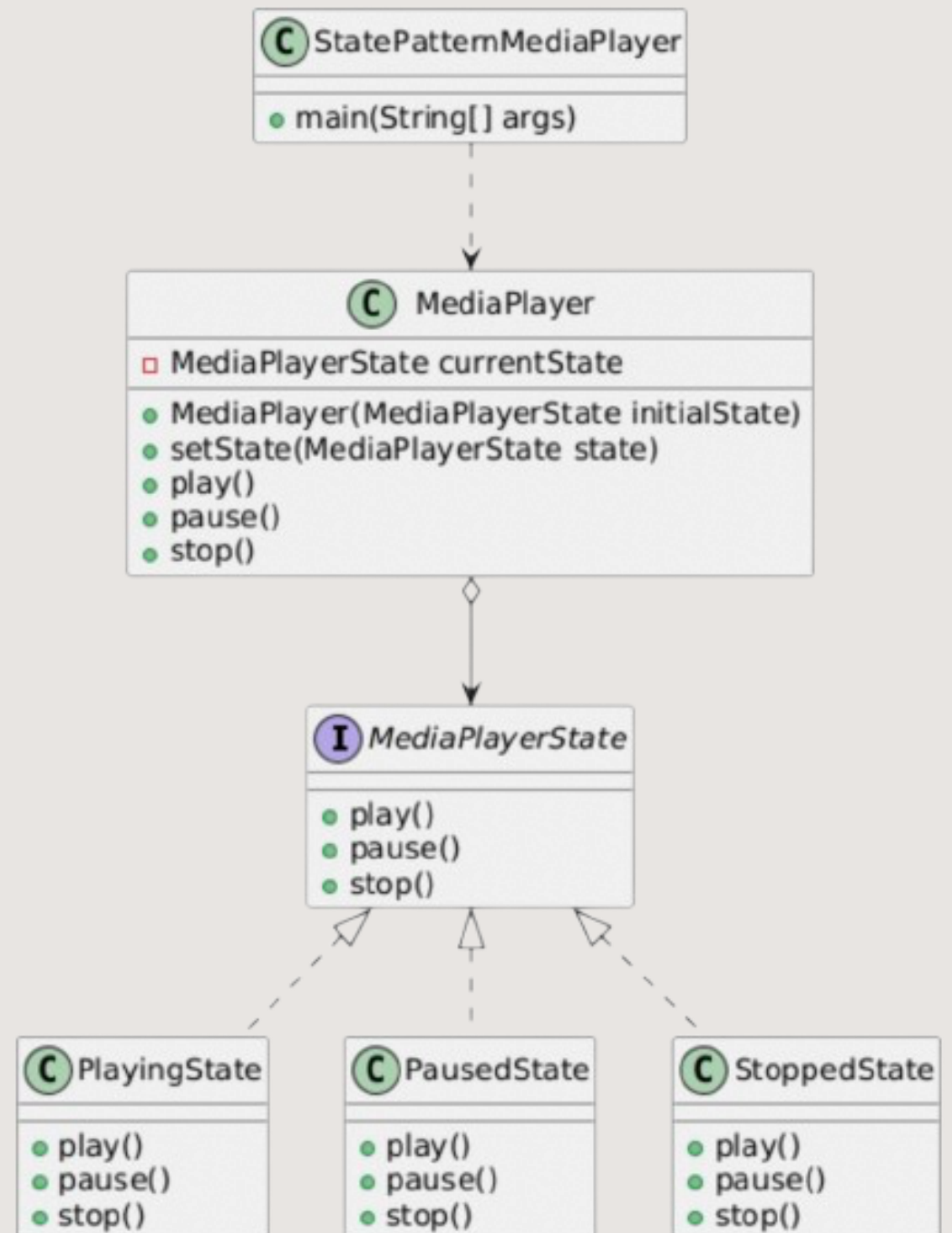


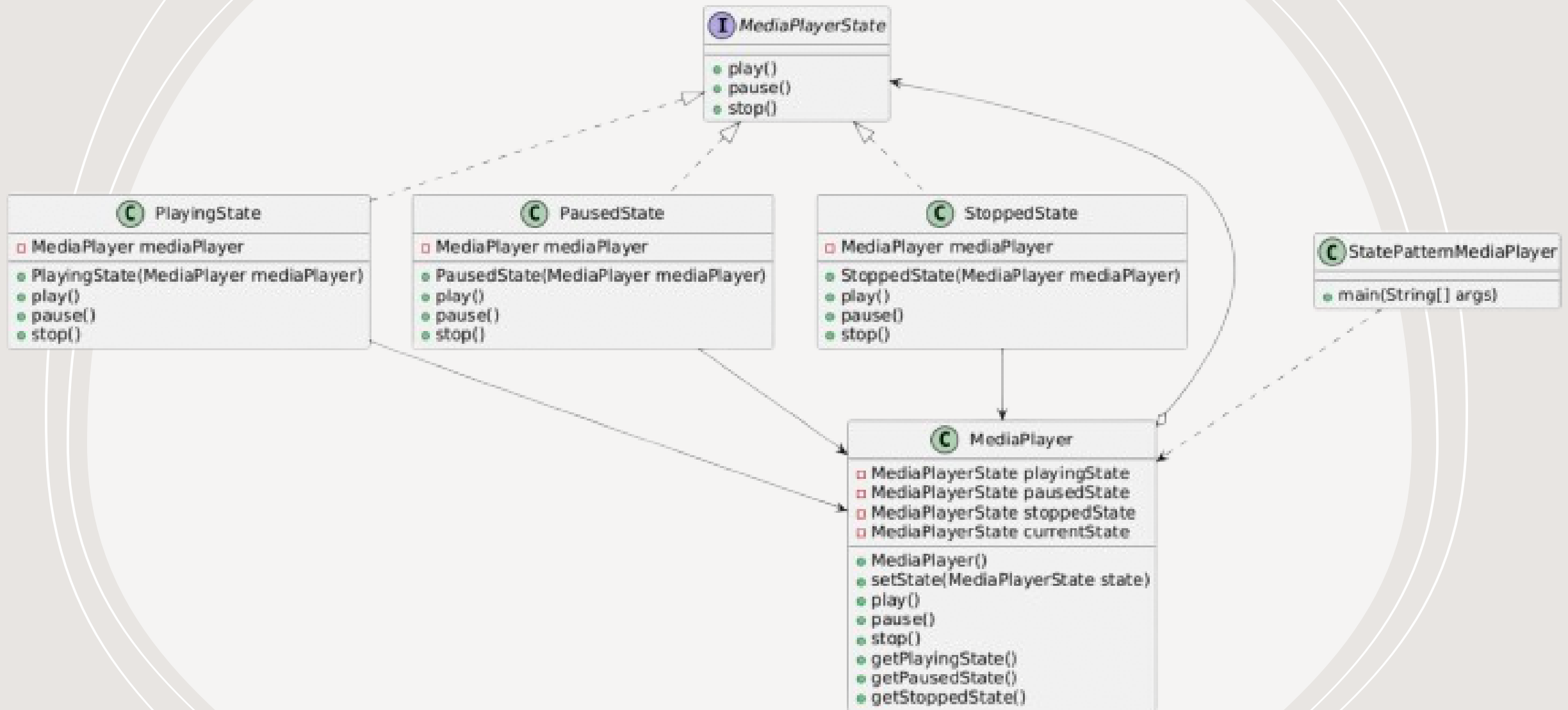
- Použití vzoru může být přehnané, pokud má stavový automat jen několik stavů nebo se mění jen zřídka.



Struktura

Implementace





Implementace

```
public interface MediaPlayerState { 2 implementations
    void play(); 2 implementations
    void pause(); 2 implementations
    void stop(); 2 implementations
}
```

```
public class PausedState implements MediaPlayerState { 1 usage
    private MediaPlayer mediaPlayer; 5 usages
    public PausedState(MediaPlayer mediaPlayer) { 1 usage
        this.mediaPlayer = mediaPlayer;
    }
    @Override
    public void play() {
        System.out.println("Pokračuji v přehrávání.");
        mediaPlayer.setState(mediaPlayer.getPlayingState());
    }
    @Override
    public void pause() { System.out.println("Již je v pauze."); }
    @Override
    public void stop() {
        System.out.println("Zastavuji přehrávání z pauzy.");
        mediaPlayer.setState(mediaPlayer.getStoppedState());
    }
}
```

```
public class PlayingState implements MediaPlayerState { 1 usage
    private MediaPlayer mediaPlayer; 5 usages
    public PlayingState(MediaPlayer mediaPlayer) { 1 usage
        this.mediaPlayer = mediaPlayer;
    }
    @Override
    public void play() { System.out.println("Již přehrávám."); }
    @Override
    public void pause() {
        System.out.println("Pauzuji přehrávání.");
        mediaPlayer.setState(mediaPlayer.getPausedState());
    }
    @Override
    public void stop() {
        System.out.println("Zastavuji přehrávání.");
        mediaPlayer.setState(mediaPlayer.getStoppedState());
    }
}
```

```
public class StoppedState implements MediaPlayerState { 1 usage
    private MediaPlayer mediaPlayer; 3 usages
    public StoppedState(MediaPlayer mediaPlayer) { this.mediaPlayer = mediaPlayer; }
    @Override
    public void play() {
        System.out.println("Začínám přehrávat.");
        mediaPlayer.setState(mediaPlayer.getPlayingState());
    }
    @Override
    public void pause() { System.out.println("Nemohu pauzovat, přehrávání je zastaveno."); }
    @Override
    public void stop() { System.out.println("Již je zastaveno."); }
}
```

```

public class MediaPlayer {
    private MediaPlayerState playingState; 2 usages
    private MediaPlayerState pausedState; 2 usages
    private MediaPlayerState stoppedState; 3 usages

    private MediaPlayerState currentState; 5 usages

    public MediaPlayer() {
        this.playingState = new PlayingState( mediaPlayer: this);
        this.pausedState = new PausedState( mediaPlayer: this);
        this.stoppedState = new StoppedState( mediaPlayer: this);

        this.currentState = stoppedState; // Výchozí stav
    }

    public void setState(MediaPlayerState state) { 1 usage
        this.currentState = state;
        System.out.println("Stav byl změněn na: " + state.getClass().getSimpleName())
    }

    public void play() {
        currentState.play();
    }

    public void pause() { currentState.pause(); }

    public void stop() { currentState.stop(); }

    public MediaPlayerState getPlayingState() { return playingState; }

    public MediaPlayerState getPausedState() { return pausedState; }

    public MediaPlayerState getStoppedState() { return stoppedState; }
}

```

```

public class StatePatternMediaPlayer {
    public static void main(String[] args) {
        // Inicializace MediaPlayeru
        MediaPlayer mediaPlayer = new MediaPlayer();

        // Akce ve stavu zastavení
        mediaPlayer.play(); // "Začínám přehrávat."
        // Akce ve stavu přehrávání
        mediaPlayer.pause(); // "Pauzuji přehrávání."
        // Akce ve stavu pauzy
        mediaPlayer.stop(); // "Zastavuji přehrávání z pauzy."
        // Akce ve stavu zastavení
        mediaPlayer.pause(); // "Nemohu pauzovat, přehrávání je zastaveno."
    }
}

```



Zdroje

<https://refactoring.guru/design-patterns>

<http://voho.eu/wiki/navrhovy-vzor/>

<https://www.oodeesign.com/>